Assignment No. 04

Title: Consider a friend's network on face book social web site. Model it as a graph to represent each node as a user and a link to represent the friend relationship between them. Store data such as date of birth, number of comments for each user.
1. Find who is having maximum friends
2. Find who has post maximum and minimum comments
3. Find users having birthday in this month
Hint: (Use adjacency list representation and perform DFS and BFS traversals).

Introduction:

Graph:
A graph is a non-linear data structure used to represent relationships between objects.
It consists of:
- Vertices (Nodes) = Represent entities
- Edges = Represent connections between vertices

Types of Graphs:
1. **Undirected Graph** – A graph in which edges have no direction, meaning the connection between vertices is bidirectional.
2. **Directed Graph** – A graph in which edges have a specific direction from one vertex to another.
3. **Weighted Graph** – A graph in which each edge is assigned a numerical value.
4. **Unweighted Graph** – A graph in which edges do not have any associated weight.

Traversals:
1. BFS (Breadth First Search):
Breadth-First Search (BFS) is a graph traversal algorithm that explores vertices level by level, visiting all adjacent vertices of a node before moving to the next level.
Time Complexity = O(V+E) … V is no. of vertices and E is no. of edges

2. DFS (Depth First Search):
Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking.
Time Complexity = O(V+E) … V is no. of vertices and E is no. of edges

Pseudocodes:

1. addUser function
```
    Create new User object
    Append User to users list
    Append empty list to adjacency list
```

2. addFriend function
```
    Add v to adjacency list of u
    Add u to adjacency list of v
```

3. maxFrnds function

```
set maxFriends 0
set userIndex -1
For each user i:
    If size of adj[i] > maxFriends:
        set maxFriends = size of adj[i]
        set userIndex = i
    If userIndex ≠ -1:
Print user with maximum friends
```

4. bday function

```
Get current day and month
For each user:
   Extract day and month from dob
   If extracted day and month equal current:
   Print birthday message
```

5. minCmnts function

```
set minComments = infinity
set userIndex = -1
For each user:
     If comments < minComments:
          set minComments = comments
          set userIndex = index
Print user with minimum comments
```

6. maxCmnts function

```
set maxComments = -infinity
set userIndex = -1
For each user:
    If comments > maxComments:
        set maxComments = comments
        set userIndex = index
Print user with maximum comments
```

7. bfs function

```
Create visited array
Create empty queue
Mark start as visited
Enqueue start
While queue not empty:
      Dequeue vertex u
      Print u
      For each adjacent vertex v:
            If v not visited:
                  Mark v visited
                  Enqueue v
```

8. dfs function
    Mark start as visited
    Print start node
    For each adjacent vertex v:
        If v not visited:
            Call DFS(v)

Algorithms:

Add User -

Step 1: Accept user details (id, name, dob, comments).
Step 2: Create a User structure with given details.
Step 3: Insert the user into users vector.
Step 4: Add an empty adjacency list for this user.
Step 5: End.

AddFriend -

Step 1: Accept two user indices u and v.
Step 2: Insert v into adjacency list of u.
Step 3: Insert u into adjacency list of v.
Step 4: End.

Max Friends –

Step 1: Initialize maxFriends = 0.
Step 2: Initialize userIndex = -1.
Step 3: Traverse all users.
Step 4: Compare adjacency list size with maxFriends.
Step 5: Update maxFriends and userIndex if larger value found.
Step 6: Print user with maximum friends.
Step 7: End.

Birthday –

Step 1: Get current system date.
Step 2: Extract current day and month.
Step 3: Traverse each user in users list.
Step 4: Extract day and month from user's DOB.
Step 5: Compare with current date.
Step 6: If matched, print birthday message.
Step 7: End.

Minimum Comments –

Step 1: Initialize minComments to maximum integer.
Step 2: Initialize userIndex = -1.
Step 3: Traverse all users.
Step 4: Compare comment count.
Step 5: Update minComments if smaller value found.
Step 6: Print user with minimum comments.
Step 7: End.

Maximum Comments –

Step 1: Initialize maxComments to minimum integer.

Step 2: Initialize userIndex = -1.
Step 3: Traverse all users.
Step 4: Compare comment count.
Step 5: Update maxComments if larger value found.
Step 6: Print user with maximum comments.
Step 7: End.

DFS –

Step 1: Accept starting vertex.
Step 2: Mark the vertex as visited.
Step 3: Display the current vertex.
Step 4: Traverse all adjacent vertices.
Step 5: If adjacent vertex not visited, recursively call DFS.
Step 6: Continue until all reachable vertices are visited.
Step 7: End.

BFS –

Step 1: Accept starting vertex.
Step 2: Initialize visited array as false.
Step 3: Create empty queue.
Step 4: Mark starting vertex as visited.
Step 5: Insert starting vertex into queue.
Step 6: While queue is not empty:

- Step 6.1: Remove front vertex.
- Step 6.2: Display the vertex.
- Step 6.3: Traverse its adjacent vertices.
- Step 6.4: If any adjacent vertex is not visited:
  a. Mark it visited.
  b. Insert it into queue.

Step 7: End.

Flowcharts:

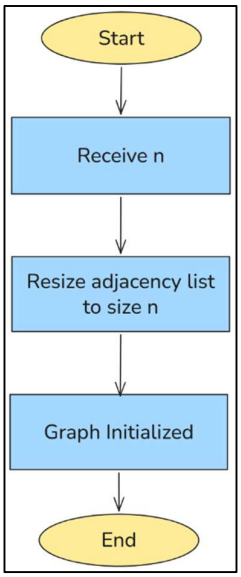

Fig 1 - Initialize Graph

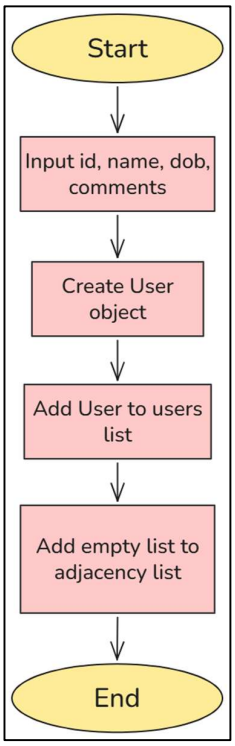

Fig 2 – Add User

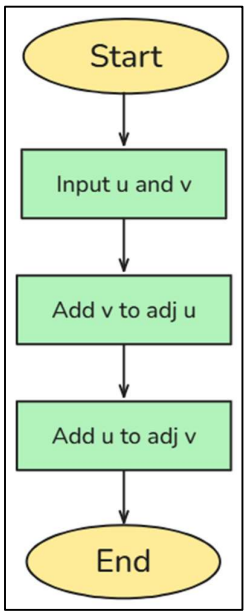

Fig 3 – Add Friend

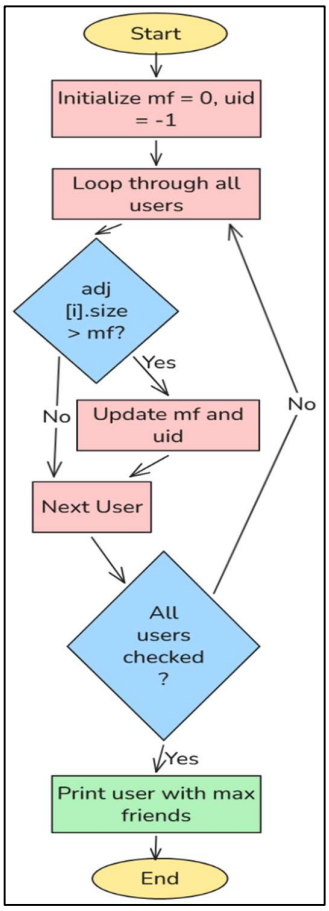

Fig 4 – Maximum Friends

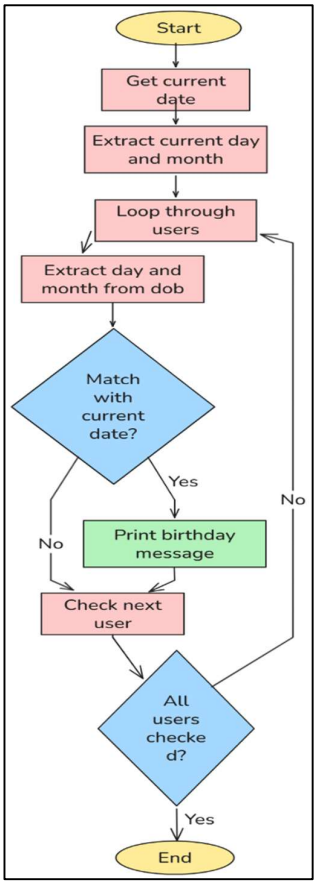

Fig 5 – Birthday of users in this month

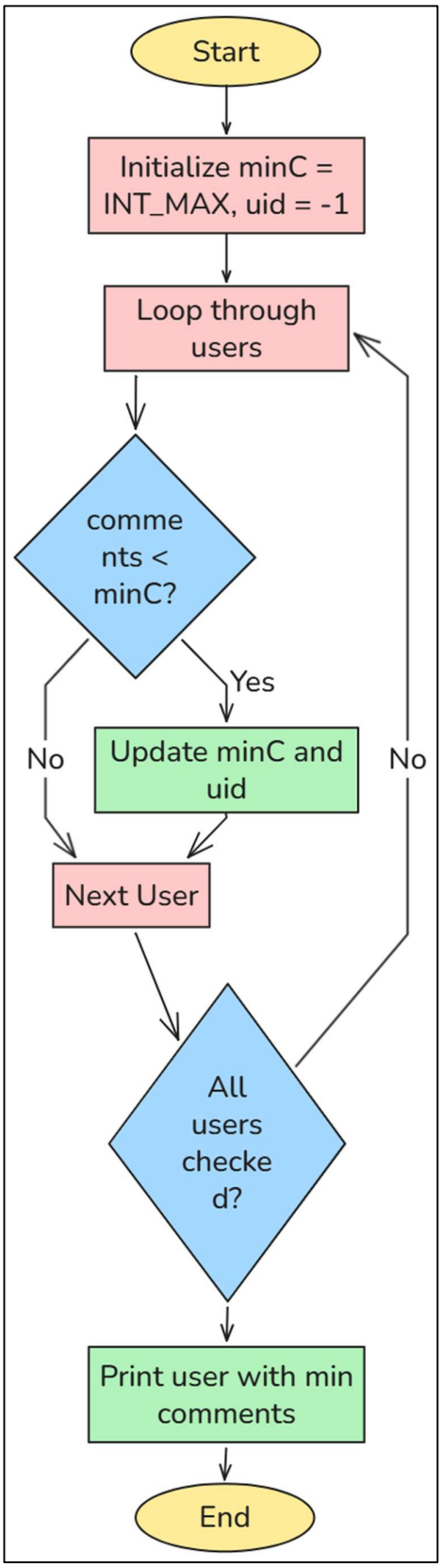

Fig 6 – Minimum Comments

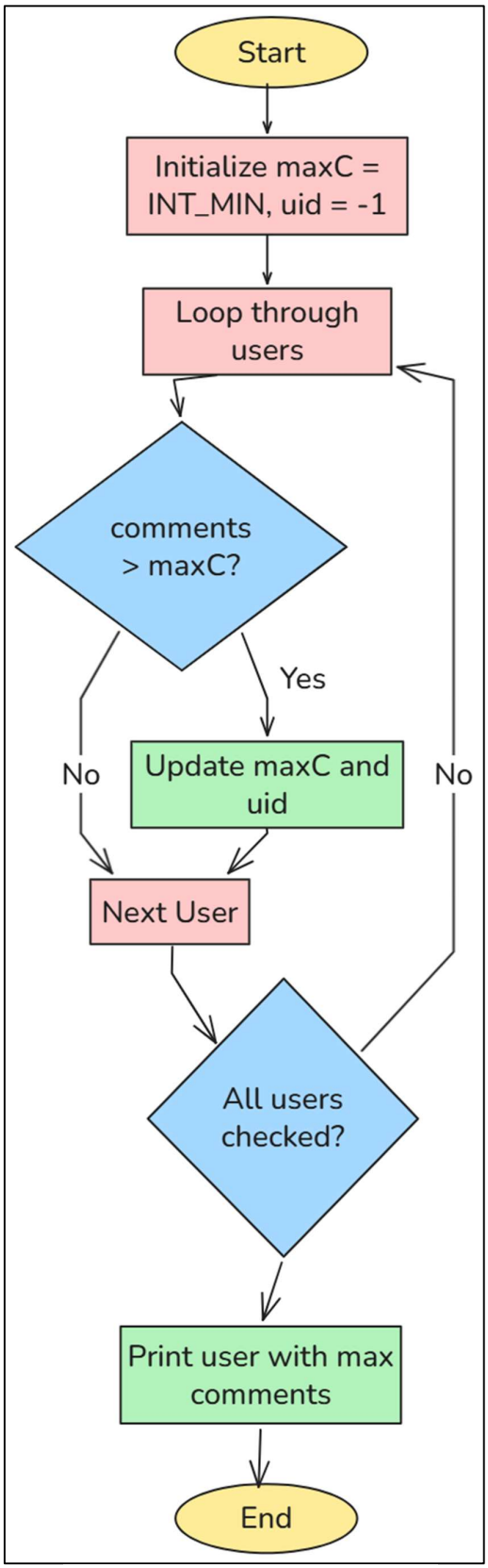

Fig 7 – Maximum Comments

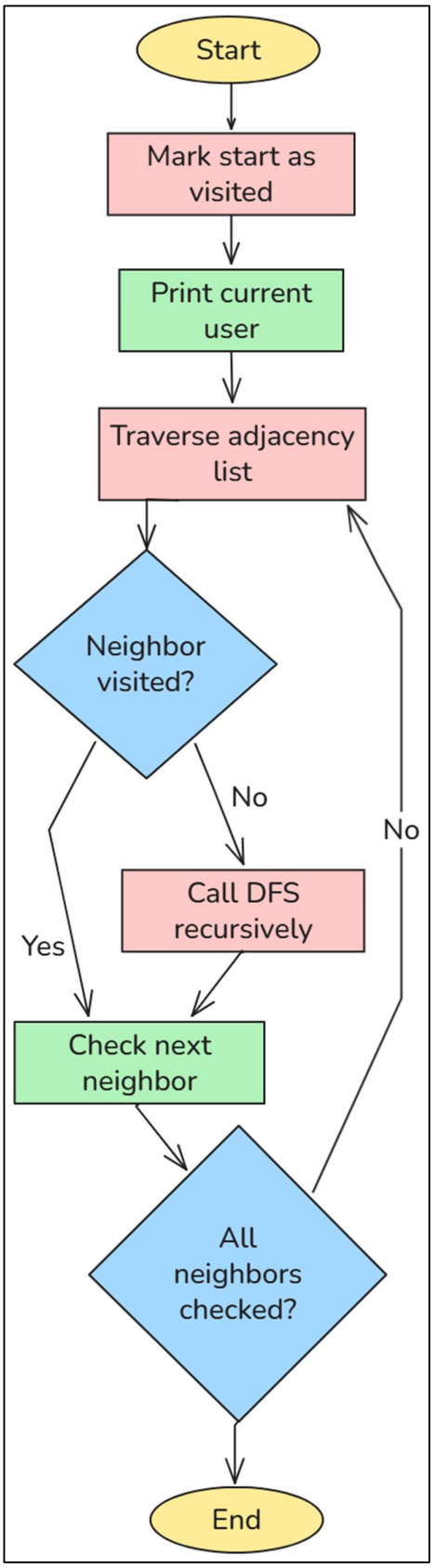

Fig 8 – Depth First Search (DFS)

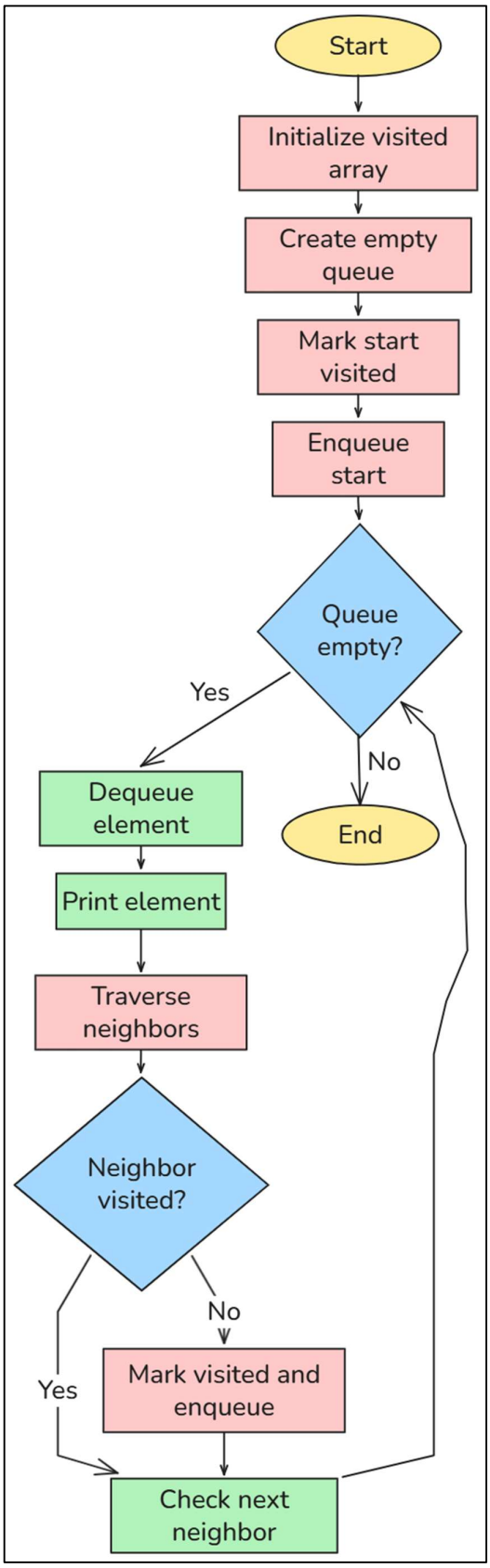

Fig 9 – Breadth First Search (BFS)

Network.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <stack>
#include <algorithm>
#include <ctime>
#include <climits>
using namespace std;

struct User{
    int id;
    string name;
    string dob;
    int comments;
};

class Graph{
    public:
        vector<User> users;
        vector<vector<int>> adj;
        Graph(int n){
            adj.resize(n);
        }
        void addUser(int id, string name, string dob, int comments){
            User u = {id, name, dob, comments};
            users.push_back(u);
            adj.push_back({});
        }
        void addFriend(int u, int v){
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
        void maxFrnds(){
            int mf=0;
            int uid=-1;
            for(int i=0;i<users.size();i++){
                if(adj[i].size()>mf){
                    mf=adj[i].size();
                    uid=i;
                }
            }
            if(uid!=-1){
                  cout<<"User with max frnds: "<<users[uid].name<<" with "<<mf<<"
frnds."<<endl;
            }
        }
        void bday(){
            time_t now = time(0);
            tm *a = localtime(&now);
            int m = a->tm_mon + 1;
            bool found = false;
            for(User u : users){
                string dob = u.dob;
                int mm = stoi(dob.substr(3,2));
                if(mm == m){
                    cout<<u.name<<"'s Birthday this month"<<endl;
                    found = true;
```

```
                }
            }
            if(!found){
                cout << "No Birthdays this month" << endl;
            }
        }
        void minCmnts(){
            int minC = INT_MAX;
            int uid = -1;
            for(int i=0;i<users.size();i++){
                if(users[i].comments<minC){
                    minC=users[i].comments;
                    uid=i;
                }
            }
            if(uid!=-1){
               cout<<"User with min comments: "<<users[uid].name<<" with "<<minC<<"
comments."<<endl;
            }
        }
        void maxCmnts(){
            int maxC = INT_MIN;
            int uid = -1;
            for(int i=0;i<users.size();i++){
                if(users[i].comments>maxC){
                    maxC=users[i].comments;
                    uid=i;
                }
            }
            if(uid!=-1){
               cout<<"User with max comments: "<<users[uid].name<<" with "<<maxC<<"
comments."<<endl;
            }
        }
};

void dfs(int start, vector<bool>& visited, Graph& g){
    visited[start]=true;
    cout<<"Visited user: "<<g.users[start].name<<endl;
    for(int a:g.adj[start]){
        if(!visited[a]){
            dfs(a, visited, g);
        }
    }
}

void bfs(int start, Graph& g){
    vector<bool> visited(g.users.size(),false);
    queue<int> q;
    q.push(start);
    visited[start]=true;
    while(!q.empty()){
        int u=q.front();
        q.pop();
        cout<<"Visited user: "<<g.users[u].name<<endl;
        for(int a:g.adj[u]){
            if(!visited[a]){
                visited[a]=true;
                q.push(a);
```

```
            }
        }
    }
}

int main(){
    Graph g(5);
    g.addUser(0, "Apurv", "28/06/2006", 10);
    g.addUser(1, "Pranet", "17/02/2006", 12);
    g.addUser(2, "Shantanu", "28/06/2006", 8);
    g.addUser(3, "XYZ", "15/08/2006", 5);
    g.addUser(4, "ABC", "01/01/2006", 8);
    g.addUser(5, "PQR", "10/03/2006", 15);

    g.addFriend(0, 1);
    g.addFriend(1, 2);
    g.addFriend(2, 3);
    g.addFriend(3, 4);
    g.addFriend(4, 5);
    g.addFriend(0, 5);
    g.addFriend(1, 3);
    g.addFriend(2, 4);

    int choice;
    bool b = true;

    while(b){
        cout << "\nSocial Media of VIT" << endl;
        cout << "1. DFS Traversal" << endl;
        cout << "2. BFS Traversal" << endl;
        cout << "3. Max Friends" << endl;
        cout << "4. Birthday Check" << endl;
        cout << "5. Min Comments" << endl;
        cout << "6. Max Comments" << endl;
        cout << "7. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        switch(choice){
            case 1: {
                cout << "\nDFS Traversal" << endl;
                vector<bool> visited(g.users.size(), false);
                dfs(0, visited, g);
                break;
            }
            case 2: {
                cout << "\nBFS Traversal" << endl;
                bfs(0, g);
                break;
            }
            case 3: {
                g.maxFrnds();
                break;
            }
            case 4: {
                g.bday();
                break;
            }
            case 5: {
```

```
                g.minCmnts();
                break;
            }
            case 6: {
                g.maxCmnts();
                break;
            }
            case 7: {
                cout << "Bye Bye" << endl;
                b = false;
                break;
            }
            default:
                cout << "Invalid choice!" << endl;
        }
    }
    return 0;
}
```

Outputs:

1. DFS Traversal

```
Social Media of VIT
1. DFS Traversal
2. BFS Traversal
3. Max Friends
4. Birthday Check
5. Min Comments
6. Max Comments
7. Exit
Enter your choice: 1

DFS Traversal
Visited user: Apurv
Visited user: Pranet
Visited user: Shantanu
Visited user: XYZ
Visited user: ABC
Visited user: PQR
```

2. BFS Traversal

```
Social Media of VIT
1. DFS Traversal
2. BFS Traversal
3. Max Friends
4. Birthday Check
5. Min Comments
6. Max Comments
7. Exit
Enter your choice: 2

BFS Traversal
Visited user: Apurv
Visited user: Pranet
Visited user: PQR
Visited user: Shantanu
Visited user: XYZ
Visited user: ABC
```

3. Users with maximum Friends

```
Social Media of VIT
1. DFS Traversal
2. BFS Traversal
3. Max Friends
4. Birthday Check
5. Min Comments
6. Max Comments
7. Exit
Enter your choice: 3
User with max frnds: Apurv with 4 frnds.
```

4. Users with birthdays in this month

```
Social Media of VIT
1. DFS Traversal
2. BFS Traversal
3. Max Friends
4. Birthday Check
5. Min Comments
6. Max Comments
7. Exit
Enter your choice: 4
PQR's Birthday this month
```

5. Users with minimum comments

```
Social Media of VIT
1. DFS Traversal
2. BFS Traversal
3. Max Friends
4. Birthday Check
5. Min Comments
6. Max Comments
7. Exit
Enter your choice: 5
User with min comments: XYZ with 5 comments.
```

6. Users with maximum comments

```
Social Media of VIT
1. DFS Traversal
2. BFS Traversal
3. Max Friends
4. Birthday Check
5. Min Comments
6. Max Comments
7. Exit
Enter your choice: 6
User with max comments: PQR with 15 comments.
```